

Software Engineering Project

DispatchNet:

Alessio Zanon

Leonardo Tonetta

Luca Fiorini

User Flow & Implementation Analysis

Contents

Purpose	2
1 TODO CLEAN CLASS DIAGRAM	3
2 TODO D3	4
2.1 Project Structure	4
2.2 Architecture	6
2.3 Implementation Language	7
2.4 Database	7
2.5 Dependencies	8
2.6 User Flows	8
2.6.1 User Management	8
2.6.2 Ticketing	10
2.6.3 Passenger	11
2.6.4 Train Company	12
2.6.5 Network Manager	13
2.7 APIs	14
2.8 Code Preview	14
2.9 Team roles	14

Readability notes

Bold words are system-important constructs.

Underlined nouns are actors of the system.

Italic words are special keywords to denote important limits of the project.

[Hyperlinks](#) in reference to sections and subsections will be blue.

[Hyperlinks](#) to external web content will be in magenta.

Purpose

This document describes all the relevant information regarding the development of Project "TRE", which implements a rail transport management system.

The purpose of this document is to:

- Present User Flows related to the actions of performed by all users.
- Describe the (Placeholder) API interface with the payment processor.
- Accompany the documentation of the implemented processes.
- Explain and justify important implementations decisions taken.
- TODO other?

Chapter 1

TODO CLEAN CLASS DIAGRAM

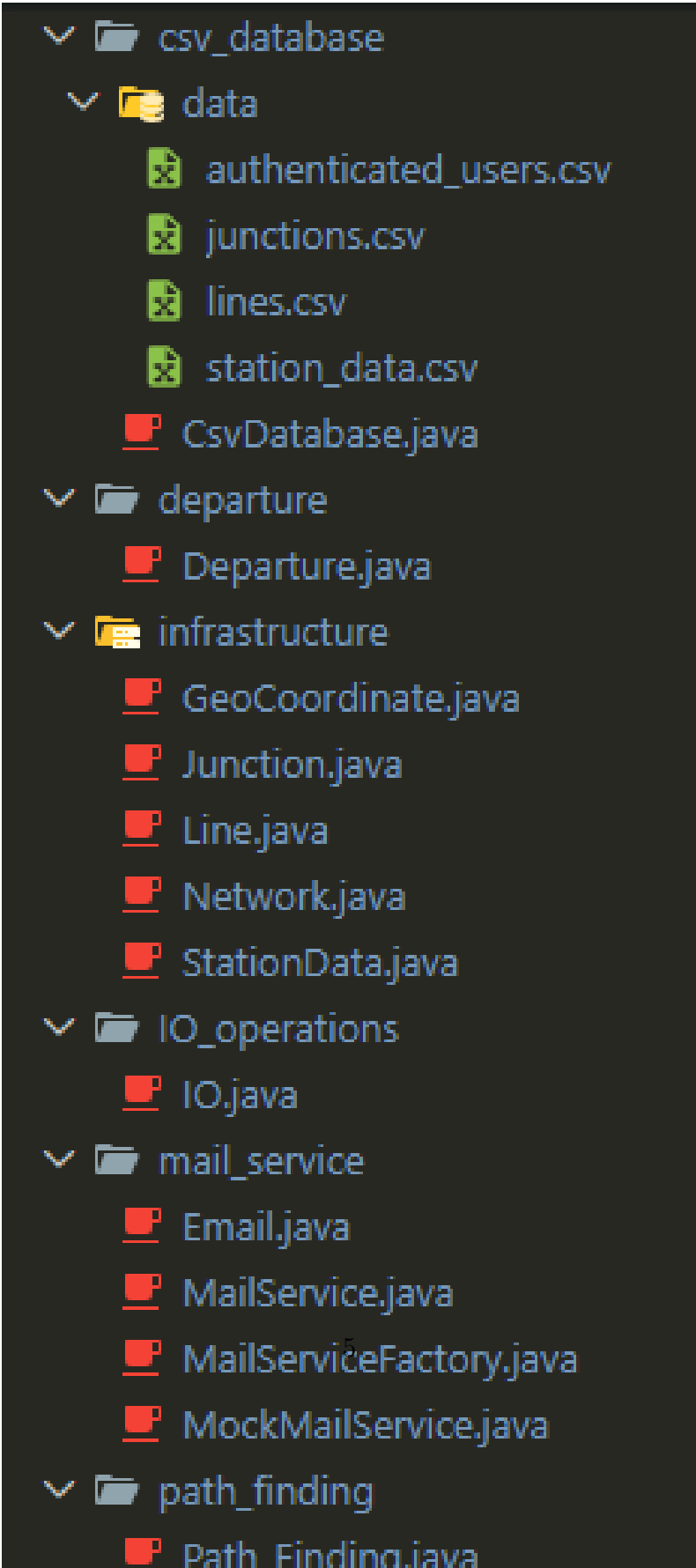
Original, up-to-date files of D1 and D2 available at our [Github here](#) and [Github here](#) respectively

Chapter 2

TODO D3

2.1 Project Structure

In the following is shown the structure of the project.



```
graph TD; csv_database[csv_database] --> data[data]; csv_database --> CsvDatabase[CsvDatabase.java]; data --> authenticated_users[authenticated_users.csv]; data --> junctions[junctions.csv]; data --> lines[lines.csv]; data --> station_data[station_data.csv]; departure[departure] --> Departure[Departure.java]; infrastructure[infrastructure] --> GeoCoordinate[GeoCoordinate.java]; infrastructure --> Junction[Junction.java]; infrastructure --> Line[Line.java]; infrastructure --> Network[Network.java]; infrastructure --> StationData[StationData.java]; IO_operations[IO_operations] --> IO[IO.java]; mail_service[mail_service] --> Email[Email.java]; mail_service --> MailService[MailService.java]; mail_service --> MailServiceFactory[MailServiceFactory.java]; mail_service --> MockMailService[MockMailService.java]; path_finding[path_finding] --> PathFinding[Path_Finding.java];
```

- ▼ csv_database
 - ▼ data
 - authenticated_users.csv
 - junctions.csv
 - lines.csv
 - station_data.csv
 - CsvDatabase.java
- ▼ departure
 - Departure.java
- ▼ infrastructure
 - GeoCoordinate.java
 - Junction.java
 - Line.java
 - Network.java
 - StationData.java
- ▼ IO_operations
 - IO.java
- ▼ mail_service
 - Email.java
 - MailService.java
 - MailServiceFactory.java
 - MockMailService.java
- ▼ path_finding
 - Path_Finding.java

Original image can be found on our [Github here](#)

It is composed of:

- the main modules:
 - departure: **TODO: ?**;
 - infrastructure: the network and its management;
 - path_finding: the algorithm to find a path between stations;
 - service_management: the management of services;
 - user_management: the management of users and their authentication.
- secondary tools:
 - csv_database: the actual database and its management;
 - IO_operations: input/output operation to interact with the user from terminal;
 - mail_service: integration with a possible mail service, as well as a temporary mock one;

2.2 Architecture

The project is intended, in its platonic ideal, as a Client-server structure, where the client shoulders only the user interface and requests to the server, who instead is responsible for the actual interfacing with the database and computing the responses.

Thanks to our chosen [Implementation Language](#), the Client may be nearly any device capable of general computing, and thus we want to minimize the client-side device requirements to fully exploit our great portability.

Thus, the client is expected to perform only to the relatively light work of displaying responses and forming requests with user input to send the central server.

Not beholden to such limitations, the central server must then receive requests and perform the computationally expensive operations, chief among

them the weighted-graph-traversal over the train network, but also loading and handling the information from the database.

Unfortunately, due to both skill and time limitations, the project is implemented as a temporary measure in a monolithic form, while still taking care to expect and enforce the necessary separation and operations for a hypothetical future division and upgrade to a client-server system ready for deployment.

2.3 Implementation Language

We chose **Java** for the operational part and **Doxygen** to generate documentation dynamically.

Our motivation for **Java** was the extreme portability and the very module-friendly structure, allowing us to experiment and tinker with different implementations of the same expected behaviour in a plug-and-play fashion. The garbage collector feature of **Java** also ensures us to spend more time working of the functioning of our systems rather than the management of their lifetimes.

Doxygen was chosen due to its in-built support for **Java** development and for its low overhead cost during code development, as the comment-tag system requires both very little time and brain-power to make, while still being very sight-readable in the code files.

2.4 Database

To save data between different we used a database saved as several CSV files. It will be loaded at startup, containing at least the account of the network manager, with default information. The data will be managed within the code and saved to the database only at the end of the program.

The database contains one file for each set of data:

- authenticated users
- junctions
- lines
- platform

- station data
- **TODO: add if necessary**

The authenticated user contains the following fields:

```
id,username,email,password,userType  
0e3c90bc-0861-4019-8e82-8b56c0275106,Admin,admin@dispatchnet.it,VerySecurePassword1234,NetworkManager
```

Original image can be found on our [Github here](#)

The junction contains the following fields:

```
id,name,latitude,longitude,stationDataId  
bbf88b8f-97cf-4c84-b43b-198cccf5a9fb,Trento,46.072194035019535,11.118837720509594,699e0587-7686-473c-9e72-70f1de3361a1
```

Original image can be found on our [Github here](#)

The line contains the following fields:

```
id,junctionId,junction2Id,lengthMeters,maxSpeedKph,nTracks  
8a4a99b0c-fc4b-4338-9af1-5568cb80b8cf,bbf88b8f-97cf-4c84-b43b-198cccf5a9fb,1ef67fd5-9fb3-4315-9125-5394ef91a75e,3100,50,1
```

Original image can be found on our [Github here](#)

The platform contains the following fields:

```
id,junctionId,name  
c4bb44b5-0d92-4705-9c83-1f614332257f,699e0587-7686-473c-9e72-70f1de3361a1,1
```

Original image can be found on our [Github here](#)

The station data contains the following fields:

```
id  
699e0587-7686-473c-9e72-70f1de3361a1
```

Original image can be found on our [Github here](#)

2.5 Dependencies

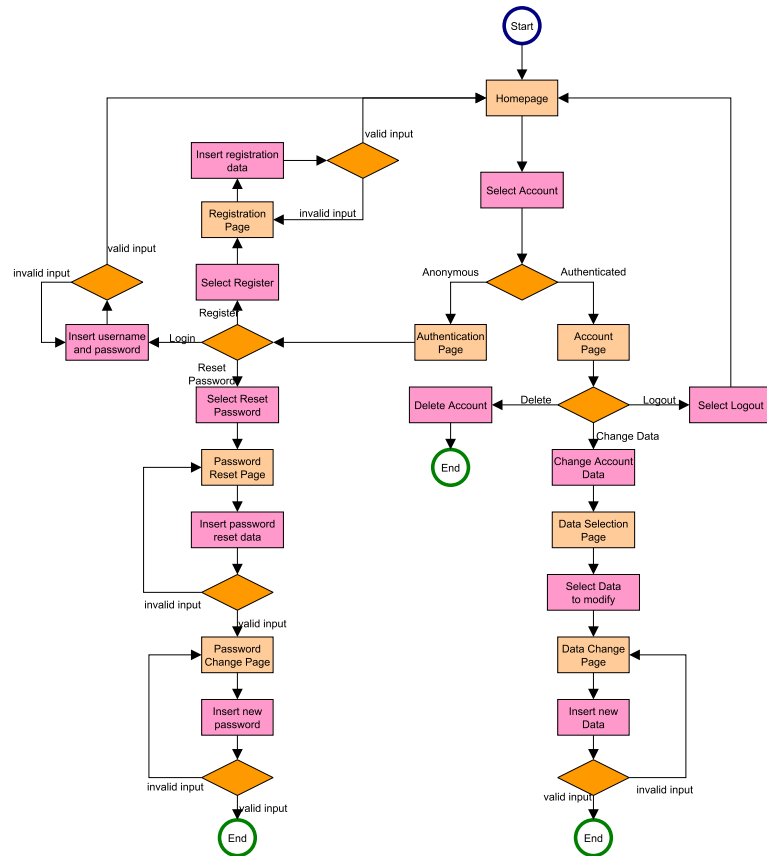
2.6 User Flows

2.6.1 User Management

The following user diagram describes the actions related to the user management and user authentication, common to all the users.

It provides different pages depending on the type of user. An authenticated user (I.E. passenger or train company, not network manager) can delete, view data (Account Page), change data or logout (also the network manager).

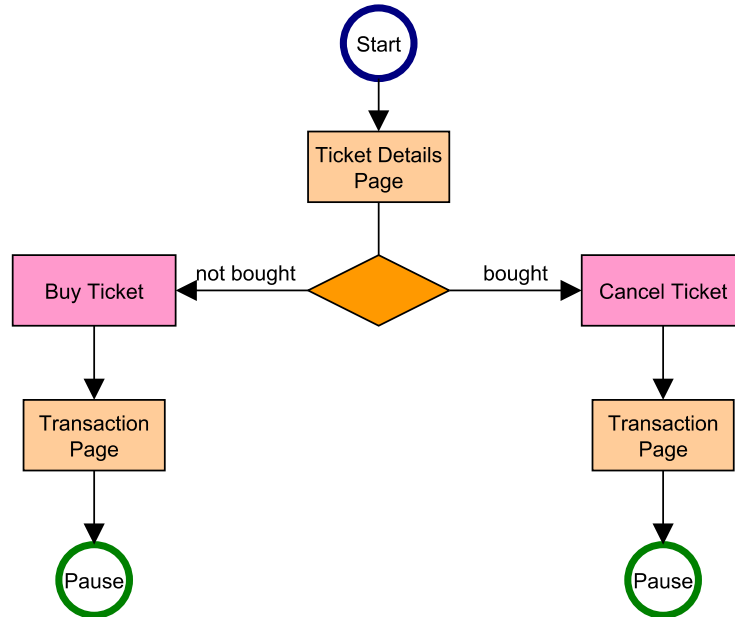
An anonymous user can register, reset its password or login.



Original image can be found on our [Github here](#)

2.6.2 Ticketing

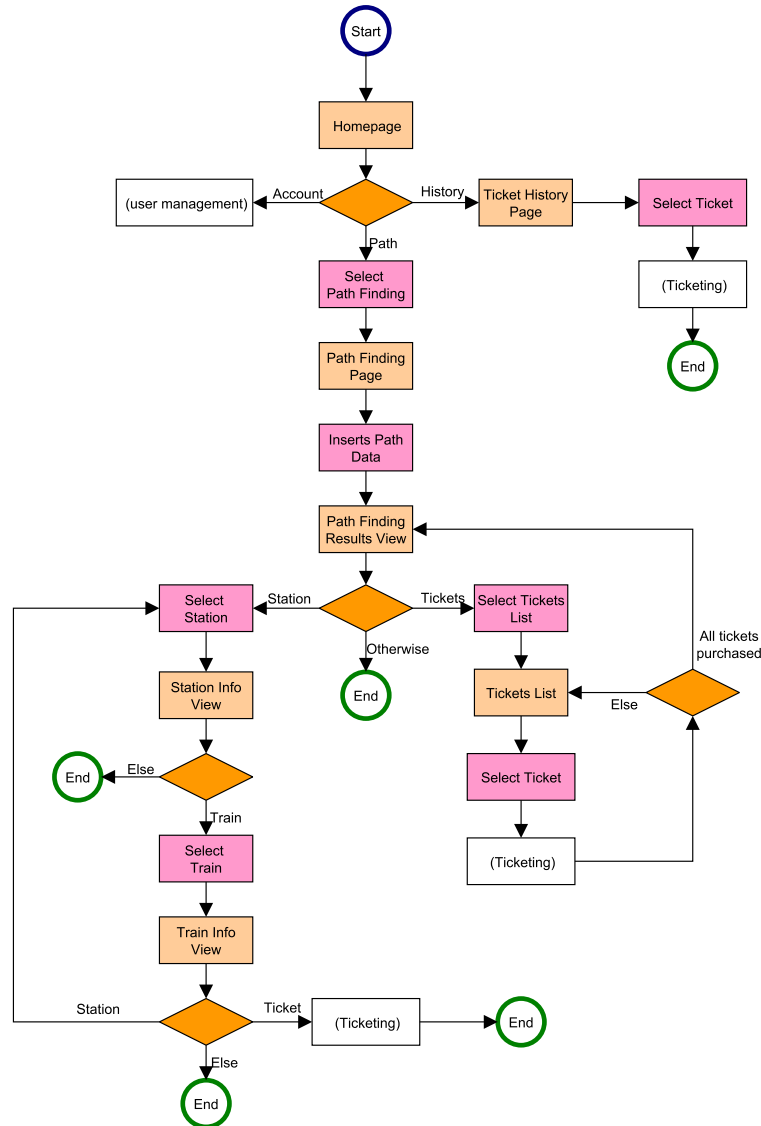
The following user diagram describes the flow of the ticketing component, started from the [Passenger](#)'s diagram.



Original image can be found on our [Github here](#)

2.6.3 Passenger

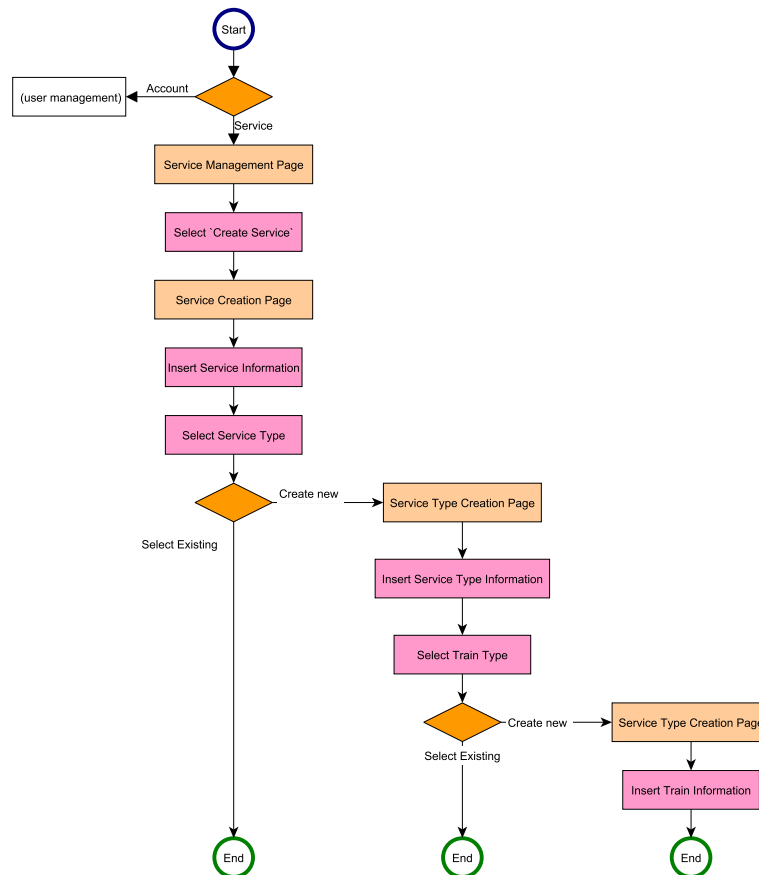
The following user diagram describes the actions the passenger can take. It can go to [User Management](#), view its ticket history or start a path finding procedure. In the last two it can move to the [Ticketing](#) flow.



Original image can be found on our [Github here](#)

2.6.4 Train Company

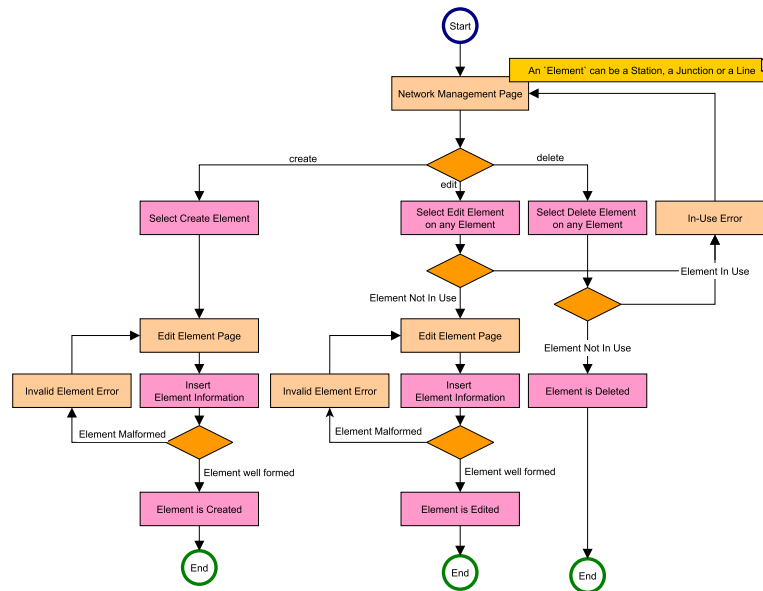
The following user diagram describes the actions the train company can take. It can go to [User Management](#) or to the service management.



Original image can be found on our [Github here](#)

2.6.5 Network Manager

The following user diagram describes the actions the network manager can take: managing the network and its elements.



Original image can be found on our [Github here](#)

2.7 APIs

As this is a mock implementation, no real API has been implemented. Instead, we opted to build fake endpoints in the form of methods, so we could test the validity of our underlying data model. All client-facing operations are found within the **UserManagement** package, in the form of class methods grouped by their relevant user. The final implementation is planned to be a REST API and will adhere to the OpenAPI standard.

External APIs have been modeled as interfaces with mock implementations. The interface paradigm allows us to use valid invocations towards a fake implementation that can easily be swapped at a later date by a real implementation. Specifically, we have implemented the following fake external APIs

- **mail_service**: The mail service
 - Interface: **MailService**
 - Mock implementation: **MockMailService**
- **payment_gateway**: The payment gateway
 - Interface: **PaymentGateway**
 - Mock implementation: **MockPaymentGateway**

The mock implementation of payment gateway also has non-deterministic behavior allowing us to test adequate handling of real-world failure conditions.

2.8 Code Preview

2.9 Team roles